

Kali Linux — Installation & First Launch

What is Kali Linux?

Kali Linux is a Debian-based Linux distribution designed mainly for **cybersecurity, penetration testing, security auditing, digital forensics, and network analysis.**

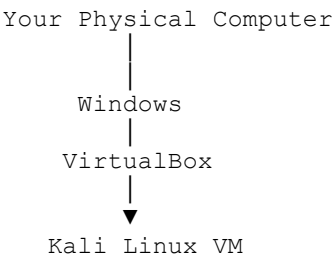
Kali Linux → Linux + Networking + Security → Cloud Security

Why Use Kali Linux in a Virtual Machine?

Instead of installing Kali directly on your computer, you can install it inside **VirtualBox**.

A Virtual Machine (VM) is basically a computer running **inside your existing computer.**

For example:



Your Windows system remains your main operating system, while Kali runs inside VirtualBox.

Advantages

- Safe learning environment
- No need to replace Windows
- Easy to delete Kali
- Easy to create snapshots
- Can run multiple operating systems
- Excellent for lab practice

What Do You Need?

You need:

Hardware

Ideally:

Component	Recommended for Kali VM
CPU	4+ cores
RAM	8 GB minimum
Storage	40–60 GB free
Virtualization	Enabled
OS	Windows/Linux/macOS

If your physical computer has **8 GB RAM**, Kali can still run, but you should avoid giving too much RAM to the VM.

If your computer has **16 GB RAM**, Kali will run much more comfortably.

5. Download VirtualBox

Download **Oracle VirtualBox** from its official website:

[VirtualBox Official Website](https://www.virtualbox.org/)

Install VirtualBox normally.

During installation, you can generally keep the default options.

6. Download Kali Linux

Go to the official Kali Linux website:

[Kali Linux Official Downloads](https://www.kali.org/downloads/)

You will see different download options.

For VirtualBox, the easiest option for beginners is the **pre-built VirtualBox image**.

Look for:

Kali Virtual Machines → VirtualBox

You will typically get an archive containing files such as:

```
.kali-vbox  
.vdi
```

The exact filenames can change between releases.

Important

Do **not** confuse:

Kali Virtual Machine Image

with:

Kali Installer ISO

For your first VirtualBox installation, I recommend the **pre-built VirtualBox VM image**, because you don't have to manually install Kali from an ISO.

Install Kali in VirtualBox

After downloading Kali's VirtualBox image:

1. Extract the downloaded archive.

2. Open **VirtualBox**.
3. Select **File** → **Import Appliance**.
4. Select the Kali appliance file.
5. Click **Next**.
6. Review the VM settings.
7. Click **Import**.

VirtualBox will create the Kali VM.

Recommended VM Specifications

This is an important part.

When creating/importing the Kali VM, configure approximately:

RAM

4 GB RAM

For normal learning:

Kali VM = 4 GB RAM

If your physical machine has only **8 GB RAM**, don't allocate 6–8 GB to Kali.

Example:

Physical RAM = 8 GB

↓

Windows = ~4 GB+

↓

Kali VM = 3–4 GB

If your physical machine has **16 GB RAM**:

Kali VM = 4–6 GB

is comfortable.

CPU

Go to:

Settings → **System** → **Processor**

Recommended:

2 CPU cores

If your computer has plenty of CPU cores:

2–4 cores

is fine.

Don't allocate all CPU cores to Kali.

For example:

Physical CPU = 8 cores

Windows → needs CPU

Kali → 2-4 cores

Storage

For basic learning:

40 GB minimum

Better:

50-60 GB

If you plan to install many security tools and create labs:

60-80 GB

is more comfortable.

Recommended

50 GB dynamically allocated disk

Dynamic allocation means VirtualBox doesn't immediately consume the entire 50 GB.

Video Memory

Go to:

Settings → Display

Set:

Video Memory = 128 MB

If available, enable:

3D Acceleration

This can help with desktop performance, although Kali doesn't require a powerful GPU for normal Linux/security learning.

Network Settings

Go to:

Settings → Network

For your initial setup, select:

Attached to: NAT

What is NAT?

NAT allows your Kali VM to access the Internet through your host computer.

Conceptually:

```
Internet
  ↓
Windows
  ↓
VirtualBox NAT
  ↓
Kali Linux
```

This is the simplest option for starting.

Later, when you learn networking labs, you can learn:

```
NAT
Bridged Adapter
Host-Only Adapter
Internal Network
```

Start Kali Linux

After configuring the VM:

1. Select **Kali Linux** in VirtualBox.
2. Click **Start**.

VirtualBox will open a new window.

Kali will boot.

You should eventually see the Kali login screen/desktop.

Kali Login

If you're using the official pre-built VM image, Kali's default credentials can vary by release/image.

For current official downloads, check the credentials shown on the download page or accompanying documentation rather than assuming an old default.

After logging in, you should see the Kali desktop.

First Things to Do After Launching Kali

Open **Terminal**.

You can use:

```
Ctrl + Alt + T
```

Then check your user:

```
whoami
```

Check the hostname:

```
hostname
```

Update Kali

After connecting to the Internet, update the package information:

```
sudo apt update
```

Then upgrade installed packages:

```
sudo apt upgrade
```

You can also use:

```
sudo apt update && sudo apt upgrade
```

You may be asked for your Kali user's password.

Understand Your Kali Desktop

Before learning hacking tools, spend some time understanding the operating system.

Learn:

Desktop

```
Application Menu  
Taskbar  
Terminal  
File Manager  
Settings  
Network
```

Kali Linux File System Structure

In Kali Linux, the file system starts from the **root directory** /.

For beginners, the following directories are especially important:

```
/
├── home
├── etc
├── var
├── tmp
├── usr
└── opt
```

1. /home — User's Personal Files

The /home directory contains the **personal directories of normal users**.

For example:

```
/home
└── kali
    ├── Desktop
    ├── Documents
    ├── Downloads
    ├── Pictures
    └── Videos
```

If the username is `kali`, its home directory is:

```
/home/kali
```

What is stored here?

- Documents
- Downloads
- Pictures
- Personal files
- User-created folders
- User configuration files

Example

```
cd /home/kali
```

Remember:

/home = Personal area of normal users

2. /etc — Configuration Files

The `/etc` directory contains **system and application configuration files**.

It controls how many parts of the Linux system and its services behave.

Examples:

```
/etc/hostname
/etc/hosts
/etc/passwd
/etc/ssh/
```

What is stored here?

- System configuration
- Network configuration
- User-related configuration
- Service configuration
- Application configuration

Example

```
ls /etc
```

Remember:

/etc = Configuration files

3. /var — Variable Data and Logs

The `/var` directory stores data that **changes frequently while the system is running**.

One of its most important directories is:

`/var/log`

which contains system and application logs.

Structure:

```
/var
├── log
├── cache
├── lib
└── tmp
```

What is stored here?

- Log files
- Cache files
- Application data
- Temporary variable data

Example

```
ls /var/log
```

Remember:

/var = Frequently changing data, especially logs

4. /tmp — Temporary Files

The `/tmp` directory is used to store **temporary files and temporary data**.

Applications and users may create temporary files here while performing tasks.

Example:

```
/tmp
├── file1
├── file2
└── temporary-data
```

You can check it using:

```
ls /tmp
```

Important

Files in `/tmp` are generally considered temporary and may be removed automatically.

Remember:

`/tmp` = Temporary files

5. `/usr` — Programs and Resources

The `/usr` directory contains many **programs, libraries, documentation, and other resources** used by the operating system and applications.

Important subdirectories include:

```
/usr
├── bin
├── sbin
├── lib
└── share
```

`/usr/bin`

Contains many executable programs and commands.

`/usr/bin`

`/usr/sbin`

Contains many system-administration programs.

`/usr/sbin`

`/usr/lib`

Contains libraries required by programs.

`/usr/lib`

`/usr/share`

Contains shared resources such as:

- Documentation
- Icons
- Application data
- Other architecture-independent files

Remember:

`/usr` = Programs, libraries and shared resources

6. `/opt` — Optional Software

The `/opt` directory is commonly used for **optional or additional software packages**, especially software installed separately from the standard system packages.

For example:

```
/opt
├─ application1
└─ application2
```

It is useful for keeping certain third-party or separately installed applications in their own directories.

Example

```
ls /opt
```

Remember:

/opt = Optional/additional software

Quick Revision

Directory	Meaning	Main Purpose
/home	Home	User's personal files
/etc	Configuration	System & application configuration
/var	Variable	Logs and frequently changing data
/tmp	Temporary	Temporary files
/usr	User/System Resources	Programs, libraries & shared resources
/opt	Optional	Additional/third-party software

Easy way to remember

```
/home → My files
/etc → Configuration
/var → Logs & changing data
/tmp → Temporary files
/usr → Programs & libraries
/opt → Additional software
```

What Are Navigation Commands?

Navigation commands are commands used to **move between directories and understand your current location** in the Linux file system.

The most important navigation commands are:

1. **pwd — Print Working Directory**

It tells you **where you currently are** in the file system.

Syntax

```
pwd
```

Example

```
kali@kali:~$ pwd
/home/kali
```

This means the current directory is:

```
/home/kali
```

2. ls — List Files and Folders

The `ls` command displays the files and directories present in the current directory.

```
ls
```

Example:

```
ls
```

Output:

```
Desktop  
Documents  
Downloads  
Pictures  
Videos
```

Useful ls Options

```
ls -l
```

Shows detailed information.

```
ls -l
```

Example:

```
drwxr-xr-x 2 kali kali 4096 Documents  
-rw-r--r-- 1 kali kali 120 notes.txt
```

It can show information such as:

- File permissions
- Owner
- Group
- Size
- Date/time
- File/folder name

```
ls -a
```

Shows **hidden files and directories**.

```
ls -a
```

Linux hidden files generally start with `..`

Example:

```
.bashrc  
.profile  
.config
```

```
ls -la
```

Shows detailed information including hidden files.

```
ls -la
```

This is a very commonly used command.

3. **cd** — Change Directory

`cd` means:

Change Directory

It is used to move from one directory to another.

Example

```
cd Documents
```

You move into the `Documents` directory.

Check your location:

```
pwd
```

Output:

```
/home/kali/Documents
```

Absolute Path

An **absolute path** starts from `/`.

Example:

```
cd /home/kali/Documents
```

This tells Linux the complete location.

Think of it like a complete postal address.

Relative Path

A **relative path** starts from your current location.

Suppose you are currently in:

```
/home/kali
```

and there is a folder:

```
/home/kali/Documents
```

You can use:

```
cd Documents
```

This is a relative path.

4. `cd ..` — Parent Directory

`..` means:

Parent directory

Suppose you are here:

```
/home/kali/Documents
```

Run:

```
cd ..
```

You will move to:

```
/home/kali
```

Example:

```
/home/kali/Documents
      ↑
    current
```

```
cd ..
```

```
/home/kali
```

5. Current Directory

`.` represents the **current directory**.

For example:

```
ls .
```

means:

List the contents of the current directory.

6. `cd /` — Go to Root

To go to the root directory:

```
cd /
```

Then:

```
pwd
```

Output:

/

7. `cd ~` — Go to Home Directory

The `~` symbol represents the current user's home directory.

```
cd ~
```

For user `kali`, this normally means:

```
/home/kali
```

You can also simply type:

```
cd
```

which normally takes you to your home directory.

8. `clear` — Clear Terminal

```
clear
```

It clears the visible terminal screen.

Shortcut:

```
Ctrl + L
```

File and Folder Operations

Linux provides commands to:

- Create files
- Create directories
- Copy files
- Move files
- Rename files
- Delete files
- Read files

1. `mkdir` — Create a Directory

`mkdir` means:

Make Directory

Syntax

```
mkdir folder_name
```

Create Multiple Folders

```
mkdir server1 server2 server3
```

This creates:

```
server1  
server2  
server3
```

2. touch — Create a File

`touch` can be used to create an empty file.

Example:

```
touch notes.txt
```

Create Multiple Files

```
touch file1.txt file2.txt file3.txt
```

3. cat — Display File Content

Suppose:

```
notes.txt
```

contains some text.

Use:

```
cat notes.txt
```

It displays the content of the file.

Example:

```
cat notes.txt
```

Output:

```
Welcome to Kali Linux
```

4. echo — Write Text into a File

You can use:

```
echo "Welcome to Kali Linux" > notes.txt
```

Now:

```
cat notes.txt
```

Output:

```
Welcome to Kali Linux
```

5. cp — Copy Files

`cp` means:

Copy

Syntax

```
cp source destination
```

Example:

```
cp notes.txt backup.txt
```

This creates a copy:

```
notes.txt  
backup.txt
```

Copy a File into a Folder

```
cp notes.txt Documents/
```

Copy a Folder

For directories, use recursive copying:

```
cp -r cloud cloud_backup
```

This copies the entire `cloud` directory and its contents.

6. `mv` — Move or Rename

`mv` means:

Move

Example:

```
mv notes.txt Documents/
```

The file is moved into `Documents`.

7. Rename a File

`mv` is also used to rename files.

```
mv old.txt new.txt
```

Now:

```
old.txt
```

becomes:

```
new.txt
```

Rename a Folder

```
mv cloud cloud-lab
```

The folder `cloud` is renamed to:

```
cloud-lab
```

8. `rm` — Remove/Delete

`rm` is used to delete files.

Example:

```
rm notes.txt
```

The file is deleted.

Important

Linux commands generally do not provide the same simple recycle-bin behavior you may be accustomed to in Windows.

Therefore, be careful with:

```
rm
```

Delete a Directory

For an empty directory:

```
rmdir cloud
```

If the directory contains files, you can use:

```
rm -r cloud
```

`-r` means recursive.

`find` — Search for Files and Directories

`find` is used to search the file system.

Example:

```
find . -name "notes.txt"
```

This searches for `notes.txt` starting from the current directory.

Another example:

```
find /home/kali -name "*.txt"
```

This searches for `.txt` files inside `/home/kali`.

Most Important Commands — Quick Revision

Command	Purpose	Example
<code>pwd</code>	Show current location	<code>pwd</code>
<code>ls</code>	List files/folders	<code>ls</code>
<code>ls -l</code>	Detailed listing	<code>ls -l</code>
<code>ls -a</code>	Show hidden files	<code>ls -a</code>
<code>cd</code>	Change directory	<code>cd Documents</code>
<code>cd ..</code>	Go to parent	<code>cd ..</code>
<code>cd /</code>	Go to root	<code>cd /</code>
<code>cd ~</code>	Go to home	<code>cd ~</code>
<code>mkdir</code>	Create directory	<code>mkdir cloud</code>
<code>touch</code>	Create file	<code>touch notes.txt</code>
<code>cat</code>	Display file	<code>cat notes.txt</code>
<code>cp</code>	Copy	<code>cp a.txt b.txt</code>
<code>mv</code>	Move/rename	<code>mv a.txt b.txt</code>
<code>rm</code>	Delete file	<code>rm a.txt</code>
<code>rmdir</code>	Delete empty directory	<code>rmdir cloud</code>
<code>find</code>	Search files/folders	<code>find . -name "*.txt"</code>
<code>clear</code>	Clear terminal	<code>clear</code>

Kali Linux Notes: Viewing File Content, Text Editors & Searching Inside Files

In Kali Linux, after learning how to create and manage files, the next important step is learning how to **view, edit, and search file contents**.

1. Viewing File Content

Linux provides several commands to view the contents of a file.

The most commonly used commands are:

- ✓ `cat`
- ✓ `less`
- ✓ `more`
- ✓ `head`
- ✓ `tail`

2. `cat` — Display Complete File Content

The `cat` command is used to **display the contents of a file directly in the terminal**.

Syntax

```
cat filename
```

Example

```
cat notes.txt
```

Output:

```
Welcome to Kali Linux.  
Linux is an open-source operating system.  
Kali Linux is based on Debian.
```

Remember:

cat = Display the complete file content

It is best for **small files**.

3. less — View Large Files

less is useful for viewing **large files one screen at a time**.

```
less notes.txt
```

You can move through the file using:

↑ / ↓	Move up/down
Space	Next page
b	Previous page
q	Quit

Example

```
less /var/log/syslog
```

Remember:

less = Read large files comfortably

4. more — View File Page by Page

more also displays a file **one screen at a time**.

```
more notes.txt
```

Press:

Space	→ Next page
q	→ Quit

Difference between less and more

For general Linux usage, **less** is **more commonly preferred** because it provides more navigation features.

5. head — View Beginning of a File

The **head** command displays the **first 10 lines** of a file by default.

```
head notes.txt
```

To display a specific number of lines:

```
head -5 notes.txt
```

This displays the first **5 lines**.

Remember:

head = Beginning of file

6. tail — View End of a File

The `tail` command displays the **last 10 lines** of a file by default.

```
tail notes.txt
```

To display the last 5 lines:

```
tail -5 notes.txt
```

Remember:

tail = End of file

7. tail -f — Monitor a File in Real Time

This is particularly useful for **Linux administration and cloud servers**.

```
tail -f /var/log/syslog
```

It continuously displays new lines added to the file.

For example:

```
Application running...
User logged in...
Connection received...
```

Press:

```
Ctrl + C
```

to stop monitoring.

Cloud Connection

When troubleshooting a cloud server, administrators often monitor application or system logs in real time.

Quick Comparison

Command	Purpose
cat	Display complete file
less	View large file page by page
more	View file page by page
head	View beginning of file
tail	View end of file
tail -f	Monitor new content in real time

9. Text Editors in Kali Linux

A **text editor** is a program used to **create and modify text files**.

Common editors available on Linux include:

- ✓ nano
- ✓ vim
- ✓ vi

For beginners, start with **nano**.

10. nano — Beginner-Friendly Text Editor

Open or create a file:

```
nano notes.txt
```

If `notes.txt` does not exist, Nano will create it.

You can type:

```
Welcome to Kali Linux.
I am learning Linux commands.
```

Important Nano shortcuts

```
Ctrl + O → Save
Ctrl + X → Exit
Ctrl + W → Search
Ctrl + K → Cut line
Ctrl + U → Paste
Ctrl + G → Help
```

Save and Exit

Press:

```
Ctrl + O
```

Then press:

```
Enter
```

Then:

Ctrl + X

11. vim / vi

vi and vim are powerful command-line text editors.

Open a file:

```
vim notes.txt
```

or:

```
vi notes.txt
```

Vim has different modes, which is the most important concept to understand.

Main Modes

```
Normal Mode  
Insert Mode  
Command Mode
```

Insert Mode

Press:

```
i
```

Then type your text.

Return to Normal Mode

Press:

```
Esc
```

Save and Exit

In Normal Mode:

```
:wq
```

Then press Enter.

Exit without saving

```
:q!
```

Remember:

Nano is easier for beginners; Vim is powerful and widely used by Linux administrators and developers.

12. Searching Inside Files

Sometimes a file contains thousands of lines. Instead of reading the entire file, we can **search for specific information**.

The most important command is:

```
grep
```

13. grep — Search Text Inside Files

`grep` searches for a **specific word or pattern** inside a file.

Syntax

```
grep "word" filename
```

Example

Suppose `notes.txt` contains:

```
Kali Linux
Ubuntu Linux
Windows
Linux Server
Cloud Computing
```

Run:

```
grep "Linux" notes.txt
```

Output:

```
Kali Linux
Ubuntu Linux
Linux Server
```

Remember:

grep = Search for text inside files

14. Case-Insensitive Search

Normally:

```
grep "linux" notes.txt
```

and:

```
grep "Linux" notes.txt
```

can produce different results because Linux commands are generally **case-sensitive**.

Use `-i` for a case-insensitive search:

```
grep -i "linux" notes.txt
```

This can find:

Linux
linux
LINUX

15. Search with Line Numbers

Use:

```
grep -n "Linux" notes.txt
```

Example output:

```
1:Kali Linux
2:Ubuntu Linux
4:Linux Server
```

The number before the text represents the **line number**.

16. Search Multiple Files

You can search multiple files:

```
grep "Linux" file1.txt file2.txt file3.txt
```

17. Search All Files in a Directory

Use:

```
grep "Linux" *
```

This searches for `Linux` in files in the current directory.

For directories and subdirectories, you can use:

```
grep -r "Linux" .
```

Here:

```
-r = recursive
.  = current directory
```

So:

```
grep -r "Linux" .
```

means:

Search for "Linux" recursively from the current directory.

18. Searching for a File — find

Remember that `grep` searches **inside files**, while `find` searches for **files/directories themselves**.

Find a file by name

```
find . -name "notes.txt"
```

Find all .txt files

```
find . -name "*.txt"
```

Difference

find → Where is the file?

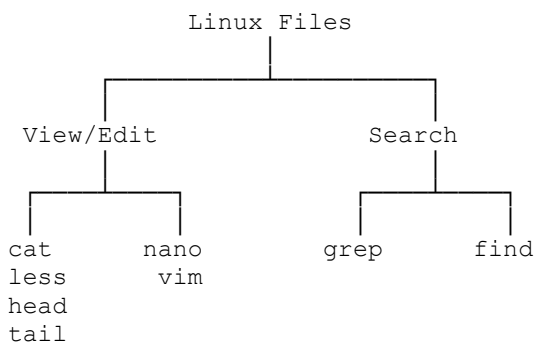
grep → What is written inside the file?

20. Important Commands — Quick Revision

Command	Use
cat file.txt	Display complete file
less file.txt	View large file
more file.txt	View file page by page
head file.txt	First 10 lines
tail file.txt	Last 10 lines
tail -f file.txt	Monitor file continuously
nano file.txt	Edit file using Nano
vi file.txt	Edit using Vi
vim file.txt	Edit using Vim
grep "word" file.txt	Search inside file
grep -i "word" file.txt	Case-insensitive search
grep -n "word" file.txt	Search with line numbers
grep -r "word" .	Search recursively
find . -name "*.txt"	Find files by name

Most Important Concept

Remember this simple distinction:



Linux Users, Groups and File Permissions

1. Terminal Shortcuts — Productivity Tips

Shortcut	Purpose
Ctrl + C	Stop/terminate the running command
Ctrl + L	Clear the terminal screen
Ctrl + D	Exit/logout from current shell
Ctrl + A	Move cursor to beginning of line
Ctrl + E	Move cursor to end of line
Ctrl + U	Delete everything before cursor
Ctrl + K	Delete everything after cursor
Ctrl + W	Delete previous word
Ctrl + R	Search command history
Tab	Auto-complete command/file/directory name
↑	Previous command
↓	Next command
Ctrl + Shift + C	Copy in terminal
Ctrl + Shift + V	Paste in terminal

Example

Suppose you previously executed:

```
sudo apt install nginx
```

Instead of typing it again, press:

```
↑
```

The previous command will appear.

Ctrl + R

If you have executed hundreds of commands and want to find an old command:

```
Ctrl + R
```

Then type:

```
nginx
```

Linux will search your command history for a command containing `nginx`.

Users in Linux

Linux is a **multi-user operating system**.

This means multiple user accounts can exist on the same Linux system.

For example:

```
Linux Computer
|
```

```
|— root
|— aman
|— rahul
|— admin
```

Each user can have different permissions.

Real-life example

Think of a company office.

There are:

- Manager
- Developer
- Accountant
- Intern

Everyone uses the same office, but everyone doesn't have access to every room.

Similarly, in Linux:

Users can access resources according to their permissions.

Root User

`root` is the **superuser/administrator account** in Linux.

Root has very high privileges and can perform administrative operations.

For example:

```
useradd aman
```

Creating users generally requires administrative privileges.

So you may use:

```
sudo useradd aman
```

Root vs Normal User

Root	Normal User
Administrator	Regular account
Very high privileges	Limited privileges
Can manage users	Normally cannot manage other users
Can change system configuration	Limited access
Can access protected resources	Access depends on permissions

Check current user

```
whoami
```

Example:

```
aman
```

Important User Commands

whoami

Shows the username of the currently logged-in user.

```
whoami
```

Example:

```
aman
```

id

Displays user ID (UID), group ID (GID), and group membership.

```
id
```

Example:

```
uid=1000 (aman) gid=1000 (aman) groups=1000 (aman),27 (sudo)
```

Important information:

UID → User ID
GID → Group ID

who

Shows users currently logged into the system.

```
who
```

w

Shows logged-in users along with information about what they are doing.

```
w
```

users

Displays usernames of currently logged-in users.

```
users
```

last

Shows login history.

```
last
```

Adding and Managing Users

Linux provides commands for creating, modifying and deleting user accounts.

The main commands are:

```
useradd
usermod
passwd
userdel
```

1. **useradd**

Used to create a new user account.

Syntax

```
useradd username
```

Example:

```
sudo useradd aman
```

This creates a user named `aman`.

Create user with home directory

```
sudo useradd -m aman
```

The `-m` option creates the user's home directory.

Usually:

```
/home/aman
```

Set password

```
sudo passwd aman
```

You will be asked to enter the password.

2. **passwd**

Used to change or set a user's password.

For your own password:

```
passwd
```

For another user as administrator:

```
sudo passwd aman
```

3. **usermod**

Used to **modify an existing user account**.

For example, adding a user to another group:

```
sudo usermod -aG developers aman
```

Here:

-a → append

-G → supplementary group

So:

Add aman to the `developers` group without removing his existing supplementary groups.

Change user's shell

```
sudo usermod -s /bin/bash aman
```

Lock a user

```
sudo usermod -L aman
```

Unlock a user

```
sudo usermod -U aman
```

4. userdel

Used to delete a user.

```
sudo userdel aman
```

To remove the user along with the home directory:

```
sudo userdel -r aman
```

Groups in Linux

A **group** is a collection of users.

Groups make permission management easier.

Real-life example

Suppose a company has:

```
Developers
├── Aman
├── Rahul
└── Priya
```

Instead of giving permissions separately to Aman, Rahul and Priya, we can give permissions to the **Developers group**.

Then all members can get the required access.

1. Creating a Group

Use:

```
sudo groupadd developers
```

groupadd creates a new group account.

Check groups:

```
groups
```

Check a particular user's groups:

```
groups aman
```

Or:

```
id aman
```

2. Adding User to Group

```
sudo usermod -aG developers aman
```

Meaning:

```
usermod → modify user  
-a → append  
-G → supplementary group  
developers → group  
aman → user
```

Important

Use:

```
-aG
```

rather than only:

```
-G
```

because `-G` can replace the user's supplementary group list, while `-a` appends the new group.

3. Removing User from Group

One common command is:

```
sudo gpasswd -d aman developers
```

This removes aman from the developers group.

File Permissions in Linux

Linux controls access to files and directories using **permissions**.

There are three main permission categories:

User/Owner
Group
Others

And three basic permissions:

r → Read
w → Write
x → Execute

1. Owner, Group and Others

Suppose we have:

`project.txt`

Its permission information might look like:

`-rw-r--r--`

Linux divides this into:

Owner	Group	Others
<code>rw-</code>	<code>r--</code>	<code>r--</code>

Meaning:

Owner

The user who owns the file.

Group

Users belonging to the file's assigned group.

Others

Everyone else who is neither the owner nor a member of that group.

2. Meaning of r, w, x

Read — r

For a file:

User can read/view its content.

Example:

`cat file.txt`

Write — w

User can modify the file.

Example:

```
nano file.txt
```

Execute — x

User can execute a file/program.

Example:

```
./script.sh
```

For directories, the meanings are slightly different:

Permission	Directory
r	List contents
w	Create/delete/rename entries
x	Enter/access the directory

3. Understanding Is -I

Use:

1s -1

Example:

```
-rwxr-xr-- 1 aman developers 1200 Aug 30 script.sh
```

Let's break it down:

```
-rwxr-xr--
```

|_____|_____

Owner Group

|

File type

More clearly:

```

-   rwx   r-x   r--
   |      |      |
   Owner  Group Others
File type

```

4. First character

- → Regular file
- d → Directory
- l → Symbolic link

5. Permissions

`rwX` → Owner
`r-X` → Group
`r--` → Others

So:

`-rwxr-xr--`

means:

- Owner → read, write, execute
- Group → read, execute
- Others → read only

Numeric Permission Values

Linux also represents permissions using numbers.

`r` = 4
`w` = 2
`x` = 1

Therefore:

`rwX` = 4 + 2 + 1 = 7
`rw-` = 4 + 2 = 6
`r-X` = 4 + 1 = 5
`r--` = 4

6. Common combinations

Number	Permission
7	<code>rwx</code>
6	<code>rw-</code>
5	<code>r-X</code>
4	<code>r--</code>
3	<code>-wx</code>
2	<code>-w-</code>
1	<code>--x</code>
0	<code>---</code>

7. chmod Command

`chmod` means:

Change Mode

It is used to change file or directory permissions.

Syntax

```
chmod permissions filename
```

8. Numeric Method

Example:

```
chmod 755 script.sh
```

Breakdown:

```
7 → Owner  
5 → Group  
5 → Others
```

Therefore:

```
755 = rwxr-xr-x
```

Meaning:

```
Owner → rwx  
Group → r-x  
Others → r-x
```

9. Another example

```
chmod 644 notes.txt
```

644 means:

```
6 → rw-  
4 → r--  
4 → r--
```

Therefore:

```
Owner → read + write  
Group → read  
Others → read
```

10. Symbolic Method of chmod

You can also use:

```
u → user/owner  
g → group  
o → others  
a → all
```

11. Add execute permission to owner

```
chmod u+x script.sh
```

12. Remove write permission from group

```
chmod g-w file.txt
```

13.Add read permission to others

```
chmod o+r file.txt
```

14.Give execute permission to everyone

```
chmod a+x script.sh
```

15.chown Command

chown means:

16.Change Owner

It changes the owner and/or group ownership of a file or directory.

Syntax

```
chown owner file
```

Example:

```
sudo chown aman file.txt
```

Now aman becomes the owner of file.txt.

17.Change Owner and Group

```
sudo chown aman:developers file.txt
```

Meaning:

Owner → aman

Group → developers

18.Change Group Only

```
sudo chown :developers file.txt
```

The owner remains unchanged, but the group becomes developers.

19.chown vs chmod

This is very important.

Command	Purpose
chmod	Change permissions
chown	Change ownership
useradd	Create user
usermod	Modify user
passwd	Set/change password
userdel	Delete user

Command	Purpose
groupadd	Create group
groups	Show group membership
id	Show UID, GID and groups
ls -l	View ownership and permissions

Simple example

Suppose:

```
file.txt
Owner: aman
Group: developers
Permission: 644
```

If you want to change:

Who owns the file?

Use:

```
chown
```

If you want to change:

What can the owner/group/others do with the file?

Use:

```
chmod
```

Quick Revision

USER MANAGEMENT

```

|— whoami      → Current user
|— id          → UID, GID, groups
|— who         → Logged-in users
|— useradd     → Create user
|— passwd     → Set/change password
|— usermod     → Modify user
|— userdel    → Delete user

```

GROUP MANAGEMENT

```

|— groupadd    → Create group
|— groups      → Show groups
|— usermod -aG → Add user to group
|— gpasswd -d  → Remove user from group

```

PERMISSIONS

```

|— r → Read
|— w → Write
|— x → Execute

```

OWNERSHIP

```
|
```

└─ chown → Change owner/group
└─ ls -l → View owner/group/permissions

PERMISSION MANAGEMENT

└─ chmod → Change permissions

Linux Basic Commands — Disk, Network, Packages, Archives & Kali Tools

1. Disk Usage Commands

Linux mein disk space check karne ke liye mainly **df** aur **du** commands use hoti hain.

df VS **du**

Command	Purpose
df	Shows available/used space of filesystems/disks
du	Shows space used by files and directories

Easy way to remember

df → How much space is available on the disk?

du → Which files/folders are using the space?

df Command

df means **disk free**.

It shows how much disk space is:

- Used
- Available
- Total

Basic command

df

Human-readable format

df -h

-h means **human-readable**.

Instead of:

10485760

you may see:

10G

Example

```
df -h
```

Output may look like:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	25G	23G	53%	/

Meaning:

Size → Total space
Used → Used space
Avail → Available space
Use% → Percentage used

Check a particular filesystem

```
df -h /
```

2. du Command

du means **disk usage**.

It tells you how much space a particular file or directory is using.

Basic

```
du
```

Human-readable

```
du -h
```

Check a directory

```
du -h /home
```

Show total size

```
du -sh /home
```

Here:

-s → Summary
-h → Human-readable

Example:

```
du -sh Downloads
```

Output:

```
5.2G    Downloads
```

This means the `Downloads` directory is using approximately **5.2 GB**.

Find size of directories

```
du -h --max-depth=1
```

This can help identify which folders are consuming space.

3. df and du — Real-Life Example

Imagine your phone has **128 GB storage**.

You want to know:

"How much storage is free?"

→ Use `df`.

```
df -h
```

Then you ask:

"Which folder is consuming most of my storage?"

→ Use `du`.

```
du -sh *
```

So:

```
df → Overall disk/filesystem space  
du → Space used by files/directories
```

4. Basic Network Commands

Linux provides several commands to check network connectivity and configuration.

Important beginner commands:

- ✓ `ip`
- ✓ `ifconfig`
- ✓ `ping`

5. ifconfig

`ifconfig` is traditionally used to display or configure network interfaces.

Basic command

```
ifconfig
```

It can show information such as:

- Network interface
- IP address
- MAC address
- Network status

Example:

```
eth0
inet 192.168.1.10
```

Here:

```
eth0 → Network interface
192.168.1.10 → IP address
```

Important Note

On many modern Linux distributions, `ifconfig` may not be installed by default. The modern command is generally:

```
ip addr
```

or:

```
ip a
```

For teaching modern Linux, prefer **ip**, while still knowing `ifconfig` because you will encounter it in older tutorials and systems.

6. ip addr

Displays IP address and network interfaces.

```
ip addr
```

Short form:

```
ip a
```

You may see:

```
inet 192.168.1.20/24
```

Here:

```
192.168.1.20 → IP address
/24 → Network prefix
```

7. ping

`ping` is used to test whether another host is reachable over the network.

Example

```
ping google.com
```

You may receive:

```
64 bytes from ...
```

This indicates that replies are being received.

Ping an IP address

```
ping 8.8.8.8
```

Stop ping

On Linux, `ping` normally continues until you stop it:

```
Ctrl + C
```

Limit number of packets

```
ping -c 4 google.com
```

`-c 4` means send **4 packets**.

8. Understanding Ping

Suppose:

```
ping google.com
```

works.

This tells us that the system can reach the destination and receive replies.

If it fails, possible reasons include:

- No internet connection
- DNS problem
- Firewall
- Destination unavailable
- Network configuration issue

So `ping` is a basic **network troubleshooting tool**.

9. APT Package Manager

Kali Linux is based on **Debian**, so it uses the **APT package management system**.

APT is used to:

- Install software
- Remove software
- Update package information
- Upgrade installed packages
- Search for packages

The main command is:

```
apt
```

You will frequently use:

```
sudo apt update
sudo apt upgrade
sudo apt install
sudo apt remove
```

10. apt update

```
sudo apt update
```

This updates the **package lists/information** from configured repositories.

Important

`apt update` does **not normally upgrade your installed software**.

It refreshes information about available packages and versions.

11. apt upgrade

```
sudo apt upgrade
```

This upgrades installed packages when newer versions are available.

Difference

```
apt update → Update package information
apt upgrade → Upgrade installed packages
```

A common sequence is:

```
sudo apt update
sudo apt upgrade
```

12. Installing Software

Use:

```
sudo apt install package-name
```

For example:

```
sudo apt install tree
```

This installs the `tree` utility.

Another example:

```
sudo apt install curl
```

General syntax

```
sudo apt install <package-name>
```

13. Removing Software

```
sudo apt remove tree
```

This removes the package.

You can also use:

```
sudo apt purge tree
```

`purge` removes the package and associated system configuration files where applicable.

14. Searching for Packages

You can search the APT package repositories:

```
apt search nmap
```

Example:

```
apt search apache2
```

This helps you find packages related to a particular name or keyword.

Installing a Tool — Practical Example

Suppose you want to install `tree`.

Step 1

Update package information:

```
sudo apt update
```

Step 2

Install:

```
sudo apt install tree
```

Step 3

Check it:

```
tree
```

Example output:

```
.
├── Documents
├── Downloads
└── Pictures
```

Archiving with `tar`

`tar` is commonly used to **create and extract archives**.

`tar` originally stands for:

Tape Archive

It can combine multiple files/directories into a single archive.

Create a .tar Archive

```
tar -cvf backup.tar Documents/
```

Meaning:

```
c → create  
v → verbose  
f → file
```

So:

```
-cvf → Create an archive file
```

17. Extract a .tar Archive

```
tar -xvf backup.tar
```

Here:

```
x → extract  
v → verbose  
f → file
```

18. Create a .tar.gz Archive

A very common format is:

```
.tar.gz
```

Create:

```
tar -czvf backup.tar.gz Documents/
```

Here:

```
c → create  
z → gzip compression  
v → verbose  
f → filename
```

Extract:

```
tar -xzvf backup.tar.gz
```

Here:

```
x → extract  
z → gzip  
v → verbose  
f → filename
```

Important

.tar → Archive
.tar.gz → Archive + gzip compression

19. zip Command

zip creates compressed ZIP archives.

Create ZIP file

```
zip backup.zip file1.txt file2.txt
```

For a directory and its contents:

```
zip -r backup.zip Documents/
```

Here:

-r → recursive

20. unzip Command

To extract a ZIP file:

```
unzip backup.zip
```

Extract to a specific directory:

```
unzip backup.zip -d /home/aman/Documents
```

Here:

-d → destination directory

21. tar vs zip

Command	Purpose
tar	Create/extract tar archives
tar -z	Work with gzip-compressed tar archives
zip	Create ZIP archives
unzip	Extract ZIP archives

Examples:

```
tar -czvf backup.tar.gz folder/  
tar -xzvf backup.tar.gz  
zip -r backup.zip folder/  
unzip backup.zip
```